



**NAME : HAMDAN ALI**

**WEEK 8 : Day 36**

**DAY 36**

**COMMAND USED IN DAY 36**

---

DAY 36 — COMMANDS USED (PowerShell / VS Code Terminal)

TaskFlow Capstone — Planning, Database Schema & Migrations

---

-----

PART 1 — Running the standalone Day 36 schema verification script

(inside Week8\Day36\verification, using raw PHP + SQLite, no

Laravel framework needed for this part)

-----

# Navigate into the verification folder

cd Day36\verification

# Run the schema verification script

php run\_schema.php

# --- Hit "could not find driver" style errors here, so checked PHP config ---



# Check which php.ini file PHP is actually loading

```
php --ini
```

# Confirm where the php.exe binary itself lives

```
Get-Command php
```

# Open php.ini in VS Code to enable the SQLite extensions

```
code C:\xampp\php\php.ini
```

# (Manually removed the leading ";" from these two lines inside php.ini:)

```
# extension=pdo_sqlite
```

```
# extension=sqlite3
```

# Search php.ini to confirm the exact line numbers/content around pdo\_sqlite

```
findstr /n "pdo_sqlite" C:\xampp\php\php.ini
```

# Verify the extensions are now actually enabled

```
php -m | findstr sqlite
```

# Re-run the script now that SQLite support is enabled

```
php run_schema.php
```

# Confirm the generated database file exists

```
dir
```

---

PART 2 — Standing up a REAL Laravel project and running the actual  
Day 36 migrations inside it (not just the raw-SQL simulation above)



-----

# Confirm Composer is installed

composer -V

# Move up one level (out of Week8) before creating the Laravel project

cd "D:\internship ( HindukushSoft Technologies )"

# First attempt — failed because a non-empty "taskflow" folder already existed

composer create-project laravel/laravel taskflow

# Inspected what was already inside that folder

dir taskflow

# Removed the old/incomplete folder

Remove-Item -Recurse -Force taskflow

# Created a fresh, real Laravel 12 project

composer create-project laravel/laravel taskflow

# Moved into the new project

cd taskflow

# Checked which migrations Laravel's default install already ran

php artisan migrate:status

# Generated a new migration to add our custom "role" column to Laravel's

# built-in users table (since our Day 36 users table has an extra column

# that Laravel's default scaffold doesn't include)

php artisan make:migration add\_role\_to\_users\_table --table=users



# Opened the migrations folder in VS Code to edit the new file

code database\migrations

# Copied in the other 4 Day 36 migrations (brand-new tables, no conflict

# with Laravel's defaults, so copied as-is)

Copy-Item "..\Week8\Day36\database\migrations\2026\_08\_24\_000002\_create\_projects\_table.php"  
"database\migrations\"

Copy-Item "..\Week8\Day36\database\migrations\2026\_08\_24\_000003\_create\_project\_user\_table.php"  
"database\migrations\"

Copy-Item "..\Week8\Day36\database\migrations\2026\_08\_24\_000004\_create\_tasks\_table.php"  
"database\migrations\"

Copy-Item "..\Week8\Day36\database\migrations\2026\_08\_24\_000005\_create\_comments\_table.php"  
"database\migrations\"

# Ran migrations — hit a "duplicate column name: role" error the first time

php artisan migrate

php artisan migrate:status

# Investigated the migration file content directly

type database\migrations\2026\_09\_04\_062259\_add\_role\_to\_users\_table.php

# Removed the broken/conflicting migration file

Remove-Item database\migrations\2026\_09\_04\_062259\_add\_role\_to\_users\_table.php

# Wiped and re-ran every migration cleanly from scratch

php artisan migrate:fresh

php artisan migrate:status

# Re-created the role migration (new timestamp) using a SIMPLE string

# column instead of enum+CHECK, since SQLite's ALTER TABLE + CHECK

# constraint combination was unreliable



```
php artisan make:migration add_role_to_users_table --table=users
```

```
# (Edited the new file's content via VS Code / heredoc — see Day37 file
```

```
# for the exact Set-Content commands used to force-write migration files)
```

```
php artisan migrate
```

```
php artisan migrate:status
```

```
# Verified the "role" column now genuinely exists on the users table
```

```
php artisan tinker --execute="print_r(Schema::getColumnListing('users'));"
```

```
# Found the "comments" table was missing — traced it to a file-content
```

```
# mixup (comments migration file accidentally contained role-migration
```

```
# code). Inspected and rewrote the file:
```

```
type database\migrations\2026_08_24_000005_create_comments_table.php
```

```
# Force-overwrote the comments migration with correct content
```

```
@"
```

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
```

```
    public function up(): void
```

```
    {
```

```
        Schema::create('comments', function (Blueprint `$table) {
```

```
            `$table->id();
```



```
`$table->foreignId('task_id')->constrained()->cascadeOnDelete();  
`$table->foreignId('user_id')->constrained()->cascadeOnDelete();  
`$table->text('body');  
`$table->timestamps();  
});  
}
```

```
public function down(): void  
{  
    Schema::dropIfExists('comments');  
}  
};
```

"@ | Set-Content -Encoding utf8 database\migrations\2026\_08\_24\_000005\_create\_comments\_table.php

# Re-verified content, then ran a clean fresh migration

type database\migrations\2026\_08\_24\_000005\_create\_comments\_table.php

php artisan migrate:fresh

# Final confirmation — full table list including "comments"

php artisan tinker --execute="print\_r(Schema::getTableListing());"

---

END OF DAY 36 COMMANDS

Result: real Laravel 12 project created; all 5 Day 36 tables (users

with role column, projects, project\_user, tasks, comments) confirmed

live and correct inside a genuine `php artisan migrate` run.

---



## EXPLANATION OF THE TASK

**DAY 35**

SOFTWARE ENGINEERING INTERNSHIP

### Week 8 — Day 36

#### Capstone Planning: Features, Database Schema & API Routes

*Project planning and database design for the internship capstone project*

---

**Project: TaskFlow — Laravel Project & Task Management System**

Submitted by: Hamdan Ali

BS Software Engineering — Air University, Islamabad

Stack: Laravel 12 · PHP 8.3 · MySQL · Blade · Tailwind CSS

## 1. What This Task Was About

Day 36 kicked off the capstone project — the biggest deliverable of the internship. Before writing a single line of application code, the brief was to sit down and actually plan the thing properly: decide what the app does, design the database so it won't need to be rebuilt halfway through, and map out the API surface. I treated this exactly the way I'd want a professional handoff document to read if I joined a company and was asked "what are we building and why."

I chose to build TaskFlow, a Laravel-based project & task management system (a lightweight Trello/Jira-style tool). I picked this over something more generic like a to-do list because it naturally forces every relationship type Laravel developers deal with day to day: one-to-many (a project has many tasks), many-to-many (users belong to many projects, projects have many users), and a self-referencing ownership pattern (a project has one owner, but many members). That range of relationships is exactly what makes a good capstone — it's small enough to finish in four days, but rich enough to actually demonstrate database design skill rather than just CRUD boilerplate.

## 2. Feature Planning

I started from the six objectives listed in the planning document (see 01-project-planning.md) rather than jumping straight to tables. Writing the objectives first — with a measurable success criterion for each one — meant every table and column I designed afterwards had a reason to exist. If I couldn't point to which objective a column served, I cut it. That's a habit I'm carrying over from the TechTarget project-planning article: plan the outcome first, then work backwards into the technical decisions.

The role model (Admin / Manager / Member) was the one design decision I went back and forth on the most. My first instinct was to put the role only on the users table. But that breaks down immediately in a multi-project tool — someone can reasonably be the manager of Project A and just a member of Project B. So the platform-level role stays on users (mainly to flag admins who can moderate anything), and the project-specific role moved onto the project\_user pivot table instead. That single decision is the difference between a schema that scales to real usage and one that quietly falls apart the moment there's more than one project per user.

## 3. Database Schema Design

The schema went through the standard DBMS pipeline: requirements → conceptual ER model → logical design → normalisation check → physical migrations. The full table-by-table breakdown is in 02-database-schema.md, and the actual migration files are under database/migrations/ in this



folder — I wrote the real Laravel migration classes at this stage rather than leaving the schema as a diagram, because in my experience a schema that hasn't been forced into an actual migration file always has a foreign-key or nullability detail you didn't think about on paper.

Five tables came out of this: users, projects, project\_user (pivot), tasks, and comments. A few decisions worth calling out:

- Enums over free-text: status and priority are constrained ENUM columns (todo/in\_progress/in\_review/done, low/medium/high) instead of plain strings, so an invalid status can never be inserted at the database layer, not just filtered out in application code.
- Cascading deletes were chosen deliberately per relationship, not applied blanket. Deleting a project cascades to its tasks (a task with no project is meaningless). Deleting a user who's assigned to a task sets assigned\_to to null instead of deleting the task (the task's work still needs doing even if the assignee left).
- A composite index on tasks(project\_id, status) — the Kanban board's main query is "give me all tasks for this project grouped by status," so that's the exact pair the index needs to cover.
- A UNIQUE constraint on project\_user(project\_id, user\_id) stops the same person being added to a project twice, which is the kind of bug that's much cheaper to prevent at the database level than to catch in application validation.

I checked the design against 1NF/2NF/3NF explicitly rather than assuming it — that's detailed in the schema document, but the short version is: no repeating groups (member lists live in their own table, not a comma-separated column), and no column depends on anything other than its own row's primary key.

## 4. API Route Planning

The route map in 03-api-routes.md was written before any controller existed, following Laravel's own RESTful resource conventions (apiResource) rather than inventing custom verbs. I used shallow nesting — /api/projects/{project}/tasks for listing and creating tasks (because the parent project matters for those two actions), but flat /api/tasks/{task} for show/update/delete, which is the pattern Laravel's documentation itself recommends once a resource has its own identity. It keeps URLs shorter without losing the parent context where it's actually needed.

Every endpoint in the plan already states which auth/role is required and which controller method will implement it, which meant Day 37 and 38 became implementation work rather than design work — the decisions were already made and written down.

## 5. Deliverables in This Folder

| File / Folder               | Purpose                                                                      |
|-----------------------------|------------------------------------------------------------------------------|
| docs/01-project-planning.md | Objectives, scope, roles, and the day-by-day milestone plan                  |
| docs/02-database-schema.md  | Full ER diagram, table specs, normalisation notes, Eloquent relationship map |
| docs/03-api-routes.md       | Complete REST API route table with auth requirements                         |
| database/migrations/*.php   | The five Laravel migrations implementing the schema above                    |

## 6. Reflection

The biggest thing I took from this task is that planning a database schema is really a design activity, not a documentation formality — the moment I forced the role model into an actual migration, the pivot-table decision above became obvious in a way it wasn't when it was just a diagram in my head. Spending a full day on planning before writing any application code felt slow at first, but it's already saved rework by Day 37, since every controller and view I write from here just has to satisfy a schema and route map that were already reasoned through.

## 7. References Consulted

- TechTarget — "Project Planning": used to structure the objectives → scope → milestone plan in the planning document.
- GeeksforGeeks — "Database Design in DBMS": used to guide the normalisation pass (1NF/2NF/3NF) on the schema.
- Video walkthroughs on project planning and database design fundamentals, watched for the overall day's learning topics.

**THANK YOU!**